# Proyecto final: Programación Científica y Técnica

## Introducción general

El proyecto final se realiza en grupo y consiste en diseñar y construir en Python una aplicación pequeña a partir de un problema del mundo real. Se evalúa la calidad del diseño y la capacidad de justificarlo, no la cantidad de código. Cada grupo elige uno de tres proyectos: **Ruta viva**, **Mercado de cartas** o **Radio infinita**.

Los enunciados son abiertos a propósito. Decidir qué entidades existen, qué responsabilidad tiene cada una y dónde vive cada regla es parte del trabajo, y es lo que se discutirá en la defensa.

### Cada grupo, su versión

Antes de programar, cada grupo define y registra su **variante**: el contexto del problema, una regla propia y un criterio de calidad, elegidos entre los ejes de variación de cada proyecto. Dos grupos del mismo proyecto no pueden tener la misma variante, y el cambio de requisito del hito 2 se asigna por grupo. Se espera que las soluciones difieran en el modelo y no solo en los nombres.

### Qué se espera

Los objetos modelan el mundo y las funciones puras calculan sobre él. El programa debe aprovechar ambas ideas y justificar dónde usó cada una. Entre otros aspectos, se evaluarán el encapsulamiento y las invariantes, la abstracción y el polimorfismo, la elección entre composición y herencia, la inmutabilidad, las funciones puras, las funciones de orden superior, la evaluación perezosa, la separación de los efectos (azar, reloj, archivos, pantalla) del cálculo y la calidad de los tests.

### Reglas de trabajo

- Se permite el uso de IA y se espera que se utilice con criterio. Cada uso se registra en `BITACORA_IA.md`: qué se pidió, qué devolvió, qué se corrigió o rechazó y por qué. Debe incluir al menos un error real de la IA.
- Solo se puede usar la librería estándar y `pytest`.
- Antes de ejecutar una función clave, el grupo anota en `PREDICCIONES.md` qué espera que devuelva, y luego compara con lo que ocurrió.
- Todos los integrantes deben poder explicar cualquier parte del código.

## Entregas

| Hito | Contenido |
|---|---|
| 1. Diseño | Variante elegida y diagrama UML de clases, con sus relaciones, interfaces y multiplicidades, junto con las decisiones de diseño y las alternativas descartadas |
| 2. Núcleo | Repositorio con el modelo, las funciones y los primeros tests. Se asigna el cambio de requisito |
| 3. Versión final | Cambio incorporado, UML actualizado, refactor documentado, bitácora completa y `ETICA.md` |
| Defensa individual | Explicar una decisión de diseño, responder sobre los conceptos y hacer un cambio en vivo, sin IA |

El UML final debe coincidir con el código. Se debe explicar qué cambió respecto del diseño original y por qué.

`ETICA.md` es una página con cuatro partes: los datos que maneja la aplicación, un riesgo ético concreto junto con el cambio de diseño que lo mitiga, un marco legal o una licencia citados después de haberlos leído, y el uso de IA realizado.

## Evaluación

| Criterio | Peso |
|---|---|
| Programación orientada a objetos (incluye el UML y su coherencia con el código) | 30 % |
| Programación funcional | 20 % |
| Cambio de requisito y refactor | 15 % |
| Defensa individual | 15 % |
| Ética y legislación | 10 % |
| Bitácora de IA | 10 % |

# Proyecto A: Ruta viva

Una persona llega a una ciudad con poco tiempo y quiere aprovecharlo. Los lugares que puede visitar son de naturaleza muy distinta: no abren en los mismos horarios, no se recorren durante el mismo tiempo, no cuestan lo mismo y algunos dependen del momento del día o del clima.

El desafío es diseñar un planificador de recorridos a pie que, dado un punto de partida, un momento del día y un tiempo disponible, proponga un recorrido. No se pide el recorrido óptimo, que es un problema muy difícil, sino un criterio explícito, explicable y fácil de cambiar.

## Preguntas de diseño

- ¿Qué tienen en común todos los lugares y qué los distingue? ¿Dónde debe vivir cada diferencia?
- ¿Qué pasa con lo ya escrito cuando aparece un tipo de lugar que nadie imaginó?
- ¿Cómo se expresa lo que le interesa a una persona de modo que pueda cambiar sin tocar el algoritmo?
- ¿Qué significa que un recorrido sea mejor que otro, y quién lo decide?
- ¿Cómo se prueba un recorrido sin salir a caminar y sin saber qué hora es?

## Restricciones

- El cálculo no puede depender del reloj del sistema ni de la ubicación real de la computadora donde se ejecuta.
- Dos recorridos calculados deben poder compararse sin que el cálculo de uno afecte al otro.
- Incorporar un tipo de lugar nuevo no debería obligar a modificar el código existente.

## Ejes de variación

Cada grupo registra una combinación y no puede repetirse dentro del proyecto. También puede proponer un eje propio, sujeto a aprobación.

- **Visitante:** familia con niños, turismo cultural, mochilero con poco presupuesto, persona con movilidad reducida, grupo nocturno.
- **Regla propia:** clima, presupuesto, transporte combinado, filas de espera, eventos con horario fijo.
- **Criterio de calidad:** variedad, mínima caminata, máxima cantidad de atractivos, equilibrio entre ellos.

### Ética y entrega

`ETICA.md` debe analizar el tratamiento de datos de ubicación (protección de datos personales, Ley 18.331), los sesgos del recorrido (comercios que pagan, personas que no pueden caminar mucho) y la licencia de los datos de mapas utilizados. Además del código y el UML, se entrega un registro de las decisiones de diseño tomadas y las alternativas descartadas.

## Proyecto B: Mercado de cartas

Un estudio independiente lanza un juego de cartas coleccionables propio y necesita una economía interna: abrir sobres, vender cartas e intercambiarlas entre jugadores. En los juegos reales, un solo bug (una carta que se duplica, monedas que aparecen de la nada) destruye la confianza de toda la comunidad.

El desafío es diseñar el sistema de modo que ciertos errores sean imposibles y no solo improbables, y que lo ocurrido sea siempre verificable y reconstruible.

### Preguntas de diseño

- ¿Qué cosas deben ser verdad siempre, pase lo que pase? ¿Cómo se garantiza que lo sean?
- Si una operación compuesta falla a la mitad, ¿en qué estado queda la economía?
- ¿Se puede saber cómo estaba todo hace cinco operaciones? ¿Y deshacer la última?
- ¿Qué cuesta tener el azar en el centro del programa? Para probar la apertura de un sobre, ¿qué hace falta?
- ¿El precio de una carta es un dato que se guarda o un resultado que se calcula?

### Restricciones

- Con los mismos datos y el mismo azar, el sistema debe producir siempre el mismo resultado.
- Una operación inválida no puede dejar rastros en el sistema.
- La moneda del juego es ficticia.

### Ejes de variación

Cada grupo registra una combinación y no puede repetirse dentro del proyecto. También puede proponer un eje propio, sujeto a aprobación.

- **Temática del juego y de sus rarezas:** criaturas, deportistas, plantas, vehículos u otra.
- **Regla económica propia:** comisiones, impuestos, ediciones limitadas, subastas, límites diarios.

- **Cómo se forma el precio:** historial de ventas, oferta y demanda, subasta, o una combinación.

### Ética y entrega

`ETICA.md` debe analizar la transparencia de las probabilidades de los sobres, la relación entre las mecánicas de azar y las apuestas (el juego de azar con dinero real está regulado por el Estado, y este mercado usa moneda ficticia) y el efecto sobre menores. Además del código y el UML, se entrega la lista de reglas que el sistema nunca debe violar, con el test que protege cada una.

## Proyecto C: Radio infinita

Una radio digital no tiene una lista de reproducción con final: en cada momento decide qué suena a continuación y aprende de cómo reacciona quien escucha.

El desafío es diseñar una radio que pueda funcionar indefinidamente, que elija según los gustos de cada oyente y que admita distintas maneras de decidir qué suena, de modo que puedan combinarse y reemplazarse sin reescribir el programa.

### Preguntas de diseño

- ¿Cómo se representa algo que nunca termina sin quedarse sin memoria?
- ¿Cómo se decide que una canción encaja? Si existen tres criterios distintos, ¿cómo se combinan sin modificar ninguno?
- ¿Qué parte del programa necesita recordar lo que pasó y cuál puede ser un cálculo puro?
- Un criterio de elección que se reduce a un solo método, ¿es una clase o una función? ¿Qué cambia según la respuesta?
- Si dos personas escuchan al mismo tiempo, ¿qué debe compartirse y qué no?
- ¿Qué pasa cuando dos canciones empatan, o cuando el resultado depende del orden del catálogo?

### Restricciones

- El flujo de canciones debe poder consumirse de a poco, sin calcularse por completo de antemano.
- Con los mismos datos, la radio debe producir siempre el mismo flujo.
- El catálogo es propio: se inventan los metadatos y no se usa audio.

## Ejes de variación

Cada grupo registra una combinación y no puede repetirse dentro del proyecto. También puede proponer un eje propio, sujeto a aprobación.

- **Tipo de emisora:** música para estudiar, para entrenar, para viajar, para una fiesta, u otra.
- **Qué señal usa para aprender:** escuchas completas, saltos, hora del día, valoraciones explícitas.
- **Regla de programación propia:** rotación de artistas, bloques temáticos, intensidad creciente, pausas cada cierta cantidad de temas.

## Ética y entrega

`ETICA.md` debe analizar el historial de escucha como dato personal que puede revelar mucho de una persona (Ley 18.331), la burbuja de filtro y el sesgo hacia lo más popular, que deja afuera a los artistas menos conocidos, y los derechos de autor sobre la música. Además del código y el UML, se entrega una tabla con cada función del núcleo que indique si es pura y, si no lo es, por qué está ahí.